

CS 253: Data Structures and Algorithms

Randomized Final Study Problem

Greedy Algorithms

Huffman Coding

Consider the string `onongigohciigojgohoo` of length 20.

- (a) Give the frequency of each character.
- (b) Build the Huffman tree. At each step remove the two trees of smallest total frequency; when frequencies tie, remove first the tree containing the alphabetically smallest character. The first tree removed becomes the left child, and left edges are labeled 0, right edges 1.
- (c) List the codeword for each character.
- (d) How many bits does the Huffman encoding of the string use? Compare this with a fixed-length code, which needs 3 bits per character.
- (e) Decode the bit string `00110101001100` using your tree.

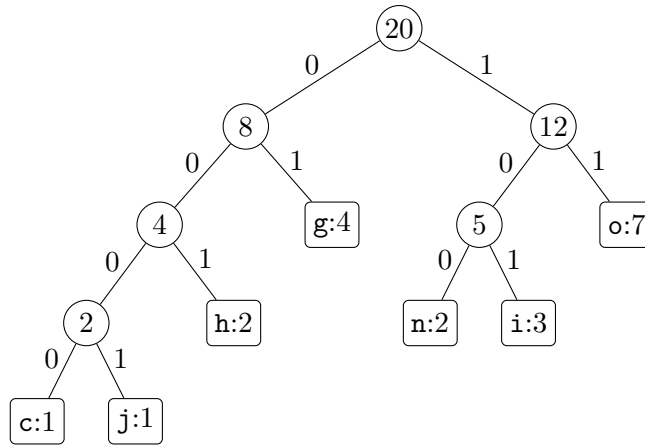
Instructor Reference

- (a) Frequencies:

Character	c	g	h	i	j	n	o
Frequency	1	4	2	3	1	2	7

- (b) Merges in order:

Step	First removed (0)	Second removed (1)	New tree
1	c (1)	j (1)	cj (2)
2	cj (2)	h (2)	chj (4)
3	n (2)	i (3)	in (5)
4	chj (4)	g (4)	cghj (8)
5	in (5)	o (7)	ino (12)
6	cghj (8)	ino (12)	cghijno (20)



(c) Codewords:

Character	Frequency	Codeword	Bits
g	4	01	$4 \times 2 = 8$
o	7	11	$7 \times 2 = 14$
h	2	001	$2 \times 3 = 6$
n	2	100	$2 \times 3 = 6$
i	3	101	$3 \times 3 = 9$
c	1	0000	$1 \times 4 = 4$
j	1	0001	$1 \times 4 = 4$

(d) Huffman encoding: 51 bits. Fixed-length: $20 \times 3 = 60$ bits, so Huffman saves 9 bits.

(e) 00110101001100 decodes to **highn**.